

Come abbiamo insegnato l'italiano a MiniCPM-o 4.5

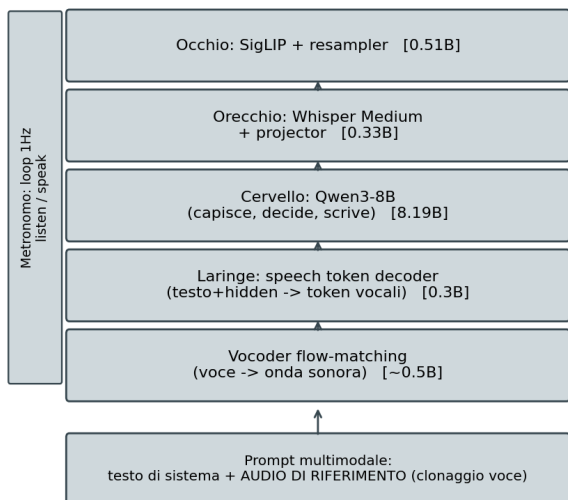
Cosa abbiamo cambiato nella struttura originale, come, e con quali risultati — spiegato in modo semplice, con gli schemi del paper originale a confronto.

Progetto: assistente vocale full-duplex in italiano su GPU locale (RTX 5090, 32 GB)

Autore del lavoro: Alessandro — documento del 2 settembre 2026

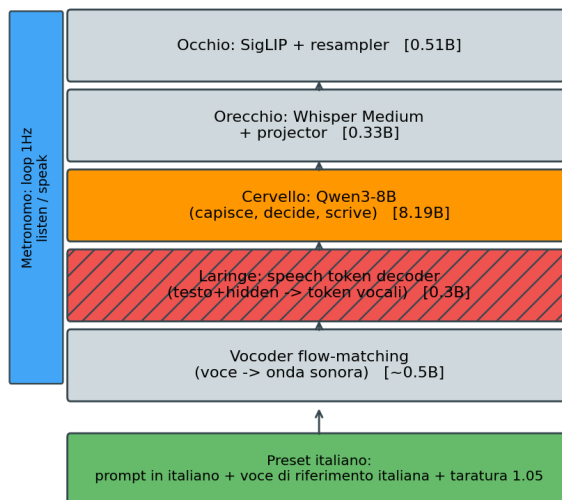
MiniCPM-o 4.5 ORIGINALE

audio in ingresso (microfono) -> ... -> audio in uscita (altoparlante)



IL NOSTRO (v1.3 italiano)

audio in ingresso (microfono) -> ... -> audio in uscita (altoparlante)



■ intatto (stock) ■ riaddestrato (LoRA fusa) ■ corretto/tarato in inferenza ■ tentato e bocciato (pron_03/04) ■ contesto (nessun peso)

In un colpo d'occhio: a sinistra il modello originale, a destra il nostro. Arancione = riaddestrato, blu = corretto in inferenza, rosso = tentato e bocciato, verde = contesto senza pesi.

1. In due parole

MiniCPM-o 4.5 è un modello «omni» da 9 miliardi di parametri che ascolta, guarda e parla **nello stesso momento** (full-duplex): ogni secondo decide da solo se stare zitto o parlare. Nasce addestrato solo su cinese e inglese. Noi lo abbiamo portato a conversare in italiano **senza cambiare la sua architettura**, agendo su quattro leve:

- **Il cervello** (Qwen3-8B, l'88% dei parametri): riaddestrato con LoRA su un corpus di conversazioni italiane e su dati di identità/ruolo, poi fuso nei pesi. Qui vive tutto il comportamento: capire, rispondere a tono, tenere il ruolo, decidere quando parlare.
- **Il metronomo** (il loop a 1 Hz che governa il tempo): struttura lasciata intatta, ma corretti quattro bug del codice originale che falsavano le decisioni di ascolto e le ripetizioni.
- **Il contesto** (prompt di sistema + audio di riferimento): il timbro «italiano» che si sente viene dal clonaggio vocale sull'audio di riferimento, non da pesi addestrati. Abbiamo costruito il preset italiano di deploy e lo abbiamo reso la fonte di verità della configurazione.
- **La laringe** (il decoder vocale da 0.3B): abbiamo provato a riaddestrarla per la pronuncia; è migliorata ma non abbastanza — è il limite aperto, e sappiamo perché (vedi §7).

Occhio, orecchio e vocoder sono rimasti quelli originali: l'orecchio (Whisper) capisce già l'italiano.

2. Il modello originale, spiegato facile

Da «a turni» a «full-duplex». I chatbot vocali classici funzionano a turni: prima ascoltano, poi parlano (e mentre parlano sono sordi). MiniCPM-o 4.5 invece percepisce in continuazione anche mentre parla, e può reagire o intervenire da solo (Figura 3 del paper).



Figura 3 del paper originale: i paradigmi a turni «bloccano» l'ingresso mentre l'AI parla; il full-duplex no.

Com'è fatto dentro (Figura 4 del paper). Immaginalo come un corpo:

- **Occhio:** SigLIP + resampler (0.51B) — comprime le immagini in 64 token per fotogramma.
- **Orecchio:** Whisper Medium + projector (0.33B) — trasforma l'audio in 10 token al secondo che il cervello capisce. Nota: *non produce mai una trascrizione*: passa direttamente «feature» continue.
- **Cervello:** Qwen3-8B (8.19B) — capisce tutto e **scrive solo testo** (3-4 token al secondo, la velocità del parlato umano). Non genera audio: per questo resta intelligente e veloce.
- **Laringe:** decoder vocale da 0.3B — riceve ogni parola del cervello *insieme al suo stato nascosto* (che porta prosodia e intenzione) e produce 25 «token vocali» al secondo.
- **Vocoder:** un decoder flow-matching trasforma i token vocali in onda sonora, imitando la voce dell'audio di riferimento (clonaggio).
- **Metronomo:** il tempo è diviso in fette da 1 secondo. A ogni fetta il modello ingerisce ciò che vede/sente e sputa PRIMA un token di controllo — **listen** (sto zitto) o **speak** (parlo) — e poi, se parla, il testo. Il paper dimostra che 1 secondo è l'ottimo e che separare «se parlare» da «cosa dire» rende l'apprendimento più stabile.

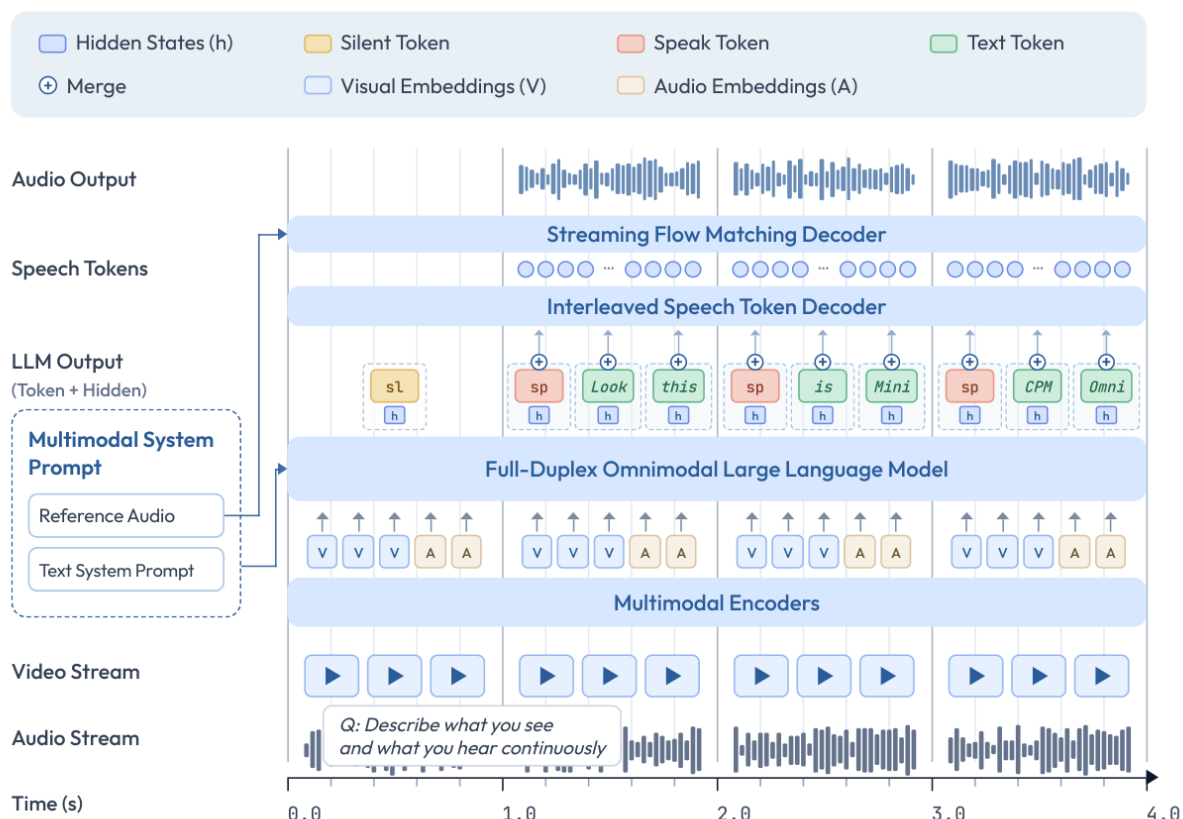


Figura 4 del paper originale: gli encoder (in basso), il cervello full-duplex (al centro), la laringe e il vocoder (in alto), tutti allineati su una linea del tempo in secondi. A sinistra il prompt multimodale con l'audio di riferimento.

Il trucco del tempo (TAIL, Figura 5). Testo e voce vengono interlacciati in modo che la voce prodotta in ogni secondo duri circa un secondo: così l'audio non «resta indietro» rispetto a ciò che il modello sta pensando ora. È questo vincolo (isocronia: 25 token vocali per secondo di unità) che ha guidato il nostro lavoro sulla densità del testo.

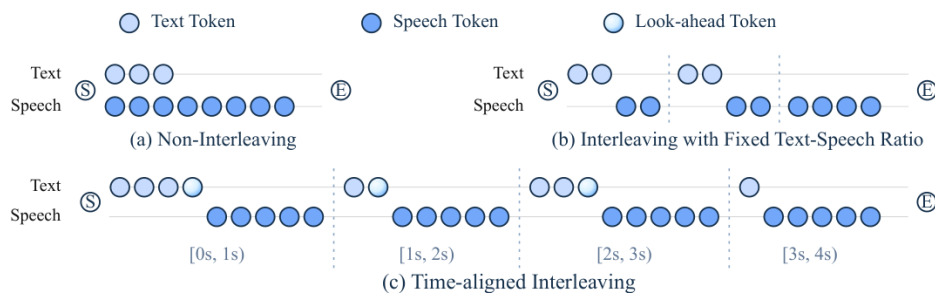
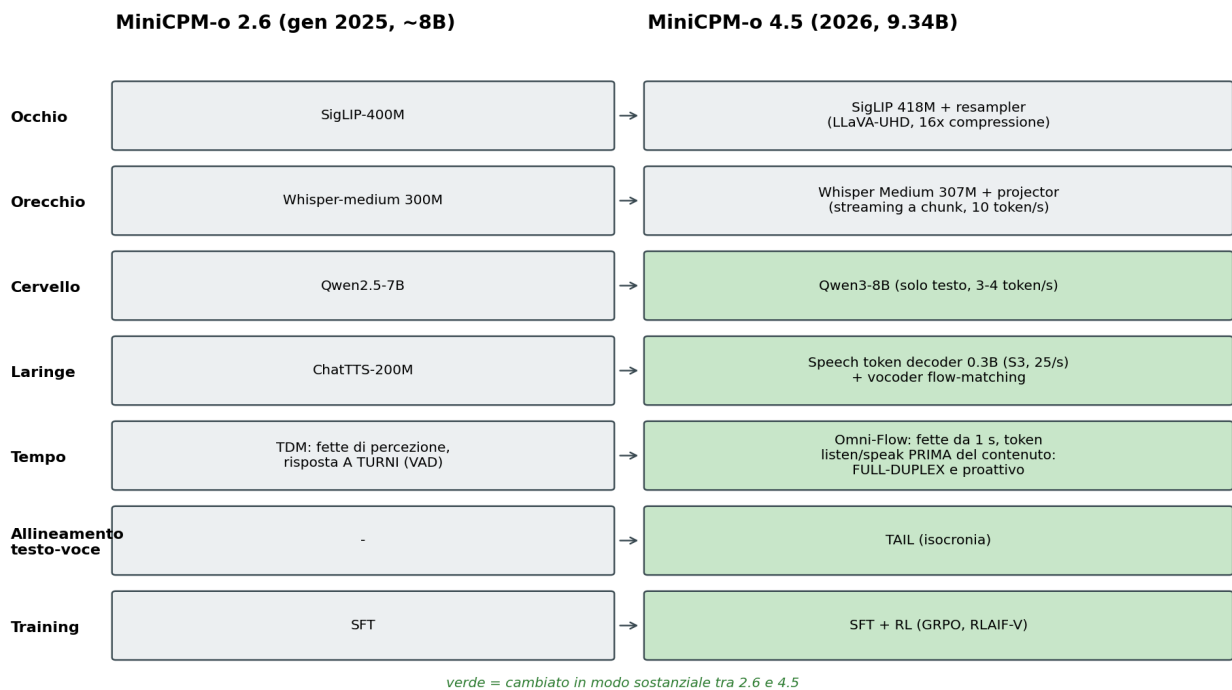


Figura 5 del paper: (a) testo troppo avanti, (b) rapporto fisso, (c) TAIL: allineato al tempo reale — la strategia di MiniCPM-o 4.5.

Cosa non c'era nel modello originale: nessuna lingua oltre cinese e inglese per la voce (il paper stesso ammette instabilità e mescolanze zh/en), nessuna valutazione dell'interruzione (la parola «barge-in» non compare), e nessuna ricetta ufficiale per riaddestrare la modalità omni/duplex (l'issue #1071 upstream lo conferma: il fine-tuning ufficiale è solo «vision»). Tutto il training che segue lo abbiamo costruito noi dal paper.

2b. In cosa è diverso da MiniCPM-o 2.6

MiniCPM-o 2.6 (gennaio 2025) è il predecessore diretto: stessa famiglia, stessa idea «omni», ma una generazione indietro su tre punti che per noi contano molto.

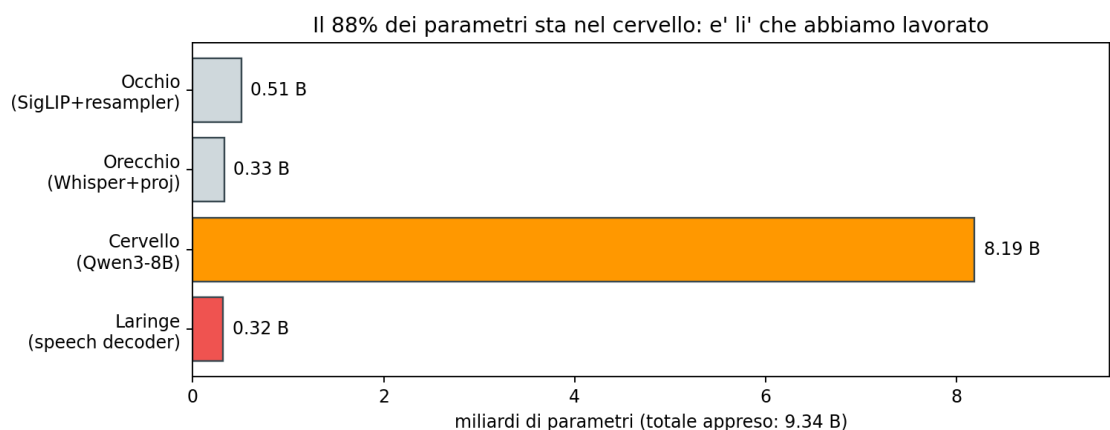


Modulo per modulo: 2.6 a sinistra, 4.5 a destra. In verde ciò che è cambiato in modo sostanziale.

Aspetto	MiniCPM-o 2.6	MiniCPM-o 4.5	Perché conta per il nostro lavoro
Conversazione	A turni : percepisce in streaming (TDM), ma risponde quando hai finito di parlare (VAD).	Full-duplex : ogni secondo decide se parlare, anche mentre sta parlando; può intervenire da solo.	Tutta la nostra pipeline dati è nel formato «unità da 1 s + token listen/speak»: esiste solo in 4.5. Su 2.6 non si trasferirebbe.
Cervello	Qwen2.5-7B	Qwen3-8B, che genera solo testo	Il cervello solo-testo rende efficace la LoRA sull'LLM senza toccare la voce.
Laringe	ChatTTS-200M	Decoder vocale 0.3B (token S3 a 25/s) + vocoder flow-matching, guidato dagli stati nascosti	È il pezzo nuovo che decide la pronuncia; sa solo zh/en in entrambe le generazioni.
Allineamento testo-voce	assente	TAIL: il testo di ogni secondo dura un secondo di voce	Il vincolo di isocronia ha guidato la nostra densità testo/unità (v1.3).
Training	SFT	SFT + RL (GRPO, RLAIF-V)	Il 4.5 arriva già più «disciplinato» sul formato; noi partiamo da lì.
Lingue voce	cinese, inglese	cinese, inglese	L'italiano manca in entrambi: il nostro lavoro è necessario in ogni caso.
Deploy	int4 ~7 GB, iPad, llama.cpp	int4 <12 GB, RTF 0.21 su RTX 4090	Entrambi girano in locale; 4.5 costa un po' di più ma resta consumer.

In una frase: 2.6 era un modello che *ascolta mentre guarda* ma parla a turni; 4.5 è il primo della famiglia che *parla mentre ascolta*. Per questo abbiamo scelto 4.5: il full-duplex nativo è proprio la capacità che volevamo in italiano, e la sua struttura (cervello solo-testo + laringe separata) è quella che rende il training mirato possibile su una GPU sola.

3. Cosa abbiamo cambiato, modulo per modulo



Dove stanno i parametri: il cervello domina. Arancione = riaddestrato, rosso = tentato e bocciato, grigio = intatto.

Modulo originale	Dimensione	Cosa gli abbiamo fatto
Occhio (SigLIP + resampler)	0.51B	Niente: stock.
Orecchio (Whisper Medium + projector)	0.33B	Niente: stock. Capisce già l'italiano (Whisper è multilingue).
Cervello (Qwen3-8B)	8.19B	Tutto il training. LoRA in più stadi, poi fusa nei pesi: comportamento in italiano, tenuta del ruolo, zero auto-dialogo, decisioni listen/speak nel contesto italiano, densità del parlato (frasi intere).
Metronomo (loop 1 Hz, listen/speak, TAIL)	meccanismo	Struttura intatta, taratura corretta: token di ascolto giusto, penalità anti-ripetizione col segno giusto (1.15), taglio del testo a confine di parola, punto operativo length_penalty 1.05.
Laringe (speech token decoder)	0.3B	Tentata (pron_03, pron_04: trainer a turno sui due canali del corpus): «più italianizzata ma non sufficiente» all'ascolto. In produzione resta quella stock (sa solo zh/en).
Vocoder (flow-matching)	~0.5B	Niente: stock.
Prompt multimodale	contesto	Preset italiano: prompt in italiano + voce di riferimento italiana + taratura. Il timbro viene dal clonaggio, non da pesi.

Il dettaglio architetturale che spiega tutto: il cervello passa alla laringe non solo il testo ma anche i suoi *stati nascosti* (sommati agli embedding del decoder vocale). Per questo riaddestrare il cervello ha migliorato anche ritmo e naturalezza del parlato senza toccare la laringe — ma non poteva correggerne la *pronuncia*: quella vive nei pesi del decoder da 0.3B, che ha imparato solo cinese e inglese.

4. Come lo abbiamo fatto: le tappe

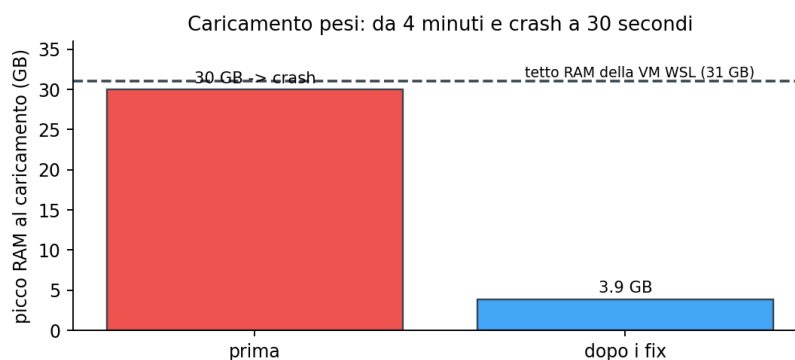
Tappa	Cosa	Perché / esito
Diagnosi	Test dal vivo del modello base con prompt italiano	Loop, eco della domanda, auto-dialogo, pronuncia cinese-inglese, parole spezzate («fac co»).
Struttura prima degli esperimenti	Lettura del paper: come è stato addestrato su ogni asse; contratto di distribuzione train-vs-deploy	Ogni corsa parte da una predizione scritta; si elimina ogni differenza tra come alleniamo e come usiamo (prompt, canali, spazi, contesto).

Tappa	Cosa	Perché / esito
Dati	Corpus italiano (~3.000 ore) copiato in locale; dataset a livello di unità da 1 s (il formato nativo del duplex) con bersagli a parole intere (v6); canali separati per ruolo; convenzione degli spazi ai confini di unità; densità 23→11-14 caratteri/unità	Il bug del mixdown mono (il modello sentiva la propria voce) e le parole incollate («chetu») nascono qui e qui sono stati risolti.
Training stadio 1 (corpus)	LoRA sul cervello, bersagli sul canale utente, 300 passi	Impara l'italiano parlato e il turn-taking reale.
Training stadio 2 (ruolo)	Mix 50/50 identità + corpus, nel contesto del preset di deploy , senza supervisione del fine-turno (a peso pieno causava mutismo)	v1 comportamentale (it08_b12): accettata dal vivo — ruolo pulito, zero loop, zero auto-dialogo.
Voce intera	Bersagli di identità reimpacchettati alla densità del parlato reale	v1.3 (it11_b12): la voce arriva a fine frase; confini di parola 100%; gate passati su 2 repliche.
Pronuncia	Trainer a turno intero per la laringe (pron_03/04, 6.000 passi)	Migliora ma non basta: serve più scala dati (vedi §7).
Codice	Audit del decoder upstream: 4 bug corretti (§5); caricamento pesi a basso consumo RAM	Loop azzerati (misurato), frammenti eliminati, boot da 4 minuti a 30 secondi.
Verifica forense	Registrazioni fedeli di ogni sessione (audio + eventi)	Hanno permesso di distinguere problemi del modello da problemi del microfono/eco (§6).

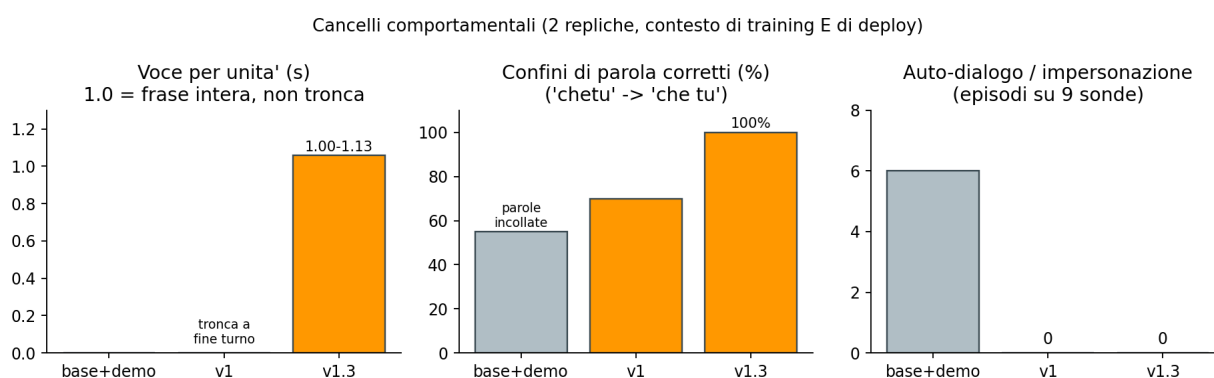
5. I bug trovati nel codice originale

Non solo pesi: leggendo il decoder upstream riga per riga abbiamo trovato errori reali che falsavano il comportamento a prescindere dal training. Il loro effetto è stato verificato con un A/B: togliendoli, loop e frammenti tornano.

Bug	Effetto	Fix
Il token di «ascolto» puntava al token sbagliato (fine-frase invece di < listen >)	Il meccanismo che favorisce l'ascolto agiva a vuoto: più difficile cedere il turno	Puntato al token corretto
Penalità anti-ripetizione con errore di segno (i token ripetuti con logit negativo venivano <i>favoriti</i>)	Loop di frasi ripetute («È tutto bene? È tutto bene?»)	Segno corretto + penalità 1.05→1.15 (misurato: azzerò i loop)
Temperatura non applicata nel campionamento della chiusura del chunk	Chiusure premature, tagli a metà parola	Temperatura applicata come da documentazione
Taglio del testo a 28 caratteri secchi, anche a metà parola	Frammentazione «forn ire», «semplice» (fuori distribuzione rispetto al training)	Taglio a confine di parola con tolleranza; token di controllo esclusi dalla cronologia della penalità
Caricamento dei pesi interamente in RAM (fp32 + pt materializzato)	Picco ~30 GB su VM WSL da 31 GB: crash ripetuti, «disconnessione» di Ubuntu	mmap + bf16 a flusso: picco 3.9 GB



6. I risultati misurati



Cancelli comportamentali: ogni candidato è valutato nel contesto di training E in quello di deploy (prompt + voce del preset); si promuove solo chi passa entrambi. Giudice finale: il test dal vivo.

- **v1.3**: voce per unità 1.00-1.13 s (frasi intere, prima si troncavano), confini di parola 100% su tutti gli indici di turno, auto-dialogo 0/9, impersonazione reale 0; holdout comportamentale: parla quando deve nell'86% dei casi (speak ratio 0.858), cede il turno nell'89% (TOR 0.891).
- **Dal vivo** (test di Alessandro): turni coerenti, ruolo che tiene, si chiude a fine frase; risposte pertinenti su domande varie. Difetti noti: pronuncia con accento cinese-inglese (laringe), qualche parola incollata ai confini di unità.
- **Diagnosi forense** (2 settembre): un episodio di «regressione» (loop, non si interrompe) è stato ricondotto con misure alle registrazioni di sessione — microfono che mandava zeri digitali a raffiche e voce dell'AI che rientrava dagli altoparlanti (correlazione 0.51-0.59 col mic) — e non al modello: con ingresso pulito il modello ascoltava 48 unità su 51 e rispondeva a tono.

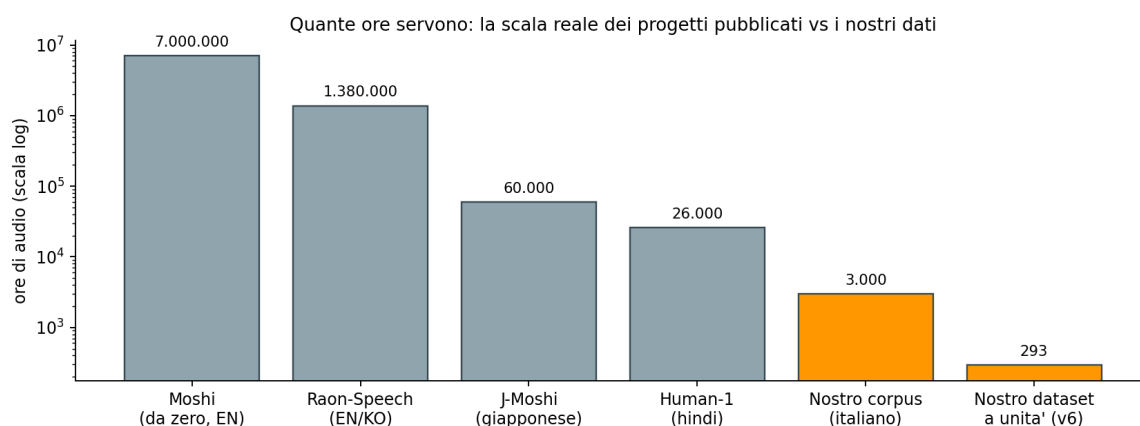
7. Cosa manca ancora, e perché

Pronuncia. Vive nella laringe (0.3B), addestrata solo su cinese/inglese. Il nostro training della laringe (pron_04) l'ha «italianizzata» ma non abbastanza: con ~300 ore di dati a unità stiamo un ordine di grandezza sotto ciò che serve per insegnare una fonetica nuova a un decoder vocale. Alternative come una TTS esterna sono state costruite e scartate per scelta architetturale (cascata).

Interruzione a metà frase (barge-in). Misurato con un A/B: il modello base originale ignora chi gli parla sopra esattamente come il nostro (22 unità di parlato di fila con la voce dell'utente nel microfono). Non è una nostra regressione: è il limite del platform, e dell'intera categoria — i benchmark 2026 (FLEXI, Full-Duplex-Bench) mostrano che perfino Moshi si ferma solo il 46% delle volte con 2.7 s di ritardo. Il «si interrompe bene» che si ottiene è turn-taking rapido (turni corti, taratura 1.05), non barge-in vero.

La scala dei dati. La letteratura che ha portato un modello duplex a una lingua nuova (J-Moshi per il giapponese, Human-1 per l'hindi) ha usato decine di migliaia di ore di conversazioni reali per il

pre-addestramento e ~1.000 ore curate per la rifinitura; il calcolo è sorprendentemente economico (~100 ore di H100 per Human-1). Il collo di bottiglia sono i dati conversazionali *veri*: sono quelli che insegnano il turn-taking, mentre i sintetici no.



Ore di audio usate dai progetti pubblicati (grigio) contro i nostri dati (arancione), scala logaritmica.

Prossimo passo naturale: replicare la ricetta Human-1 in italiano (tokenizer italiano, codec congelato, riaddestrare il transformer) espandendo il corpus con audio conversazionale italiano diarizzato — la pipeline dati costruita in questo progetto è esattamente la competenza che serve.

8. Glossario facile

Termine	Significato
Full-duplex	Ascoltare e parlare nello stesso momento, come al telefono. Il contrario è «a turni» (walkie-talkie).
End-to-end (e2e)	Un solo modello dall'audio in ingresso all'audio in uscita, senza trascrizione intermedia. Il contrario è la «cascata»: ASR → LLM → TTS, tre modelli separati.
ASR / TTS	Riconoscimento vocale (audio → testo) / sintesi vocale (testo → audio).
Unità (chunk)	La fetta di 1 secondo in cui MiniCPM-o divide il tempo. A ogni unità decide listen o speak.
Hidden state	Lo «stato mentale» interno del cervello per ogni parola: un vettore di numeri che porta significato e intenzione. Viene passato alla laringe per dare prosodia.
LoRA	Tecnica per riaddestrare un modello grande toccando pochi parametri aggiuntivi, che poi si possono fondere nei pesi originali.
Token vocale (S3)	Un «pezzetto di suono» codificato come numero; la laringe ne produce 25 al secondo, il vocoder li trasforma in onda sonora.
Isocronia	La regola per cui il testo prodotto in un secondo deve durare circa un secondo di voce (TAIL).
Gate (cancello)	Test automatico con soglia: se il candidato non lo passa, non si promuove.
Barge-in	Interrompere qualcuno mentre parla e farlo smettere.

9. Fonti

MiniCPM-o 4.5 Technical Report (paper locale in docs/minicpm-o-4.5-upstream/): architettura (Tab. 13), Omni-Flow (§3, Tab. 1), TAIL (§3.4), training (§4-5), limiti (§8).

Repository upstream OpenBMB/MiniCPM-o e issue #1071 (fine-tuning omni non supportato ufficialmente).

Storia tecnica del progetto: calendar.md, plan/audit-20agosto-fable.md, plan/contratto-distribuzione.md, plan/modus-operandi.md; releases in training/releases/ (LEGGIMI.md).

FLEXI (arXiv 2509.22243), Full-Duplex-Bench v1.5 (arXiv 2507.23159), FD-Bench (arXiv 2507.19040): benchmark 2025-26 sul barge-in.

Moshi (arXiv 2410.00037), J-Moshi (arXiv 2506.02979), Human-1 (arXiv 2604.23295), Raon-Speech (arXiv 2605.23912): ricette e scala dati per lingue nuove.

MiniCPM-o 2.6: model card openbmb/MiniCPM-o-2_6 (Hugging Face, gennaio 2025): SigLip-400M, Whisper-medium-300M, ChatTTS-200M, Qwen2.5-7B, TDM omni streaming, conversazione a turni.

Qwen3-Omni (arXiv 2509.17765) e Qwen3.5-Omni (arXiv 2604.15804): confronto architetturale (Thinker-Talker, a turni, solo API per 3.5).